

EXPERIMENT NO. 1

First Palindromic String in an Array

Programming Language: C

Name : Bodham susma

Register No. : 192571065

Aim

To write a C program that finds the first palindromic string in an array of strings, or prints an empty string if none exists.

Question

Given an array of strings words, return the first palindromic string in the array. If there is no such string, return an empty string "". A string is palindromic if it reads the same forward and backward.

Example 1

Input: words = ["abc","car","ada","racecar","cool"]

Output: "ada"

Explanation: The first string that is palindromic is "ada". Note that "racecar" is also palindromic, but it is not the first.

Example 2

Input: words = ["notapalindrome","racecar"]

Output: "racecar"

Explanation: The first and only string that is palindromic is "racecar".

Algorithm

Step 1: Start.

Step 2: Store the input strings in the array words and set foundIndex = -1.

Step 3: For each word in the array, take its length len and set left = 0, right = len - 1, isPalindrome = 1.

Step 4: While left < right, compare the characters at left and right. If they differ, set isPalindrome = 0 and stop checking this word; otherwise do left++ and right--.

Step 5: If isPalindrome is 1, save the index of the word in foundIndex and stop searching the array.

Step 6: If foundIndex is not -1, print words[foundIndex]; otherwise print an empty string.

Step 7: Stop.

Program (C)

```
#include <stdio.h>
#include <string.h>

int main() {
    // Input array of strings
    const char *words[] = {"abc", "car", "ada", "racecar", "cool"};
    int wordsSize = sizeof(words) / sizeof(words[0]);

    // Variable to store the index of the first palindrome found (-1 means none found)
    int foundIndex = -1;

    // Outer loop: Iterate through each word in the array
    for (int i = 0; i < wordsSize; i++) {
        const char *currentWord = words[i];
        int len = strlen(currentWord);

        int left = 0;
        int right = len - 1;
        int isPalindrome = 1; // Assume it is a palindrome initially (1 = true)

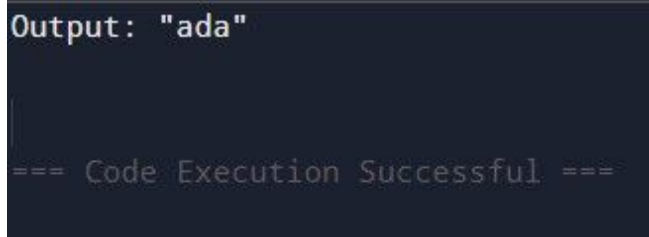
        // Inner loop: Check if the current word is a palindrome
        while (left < right) {
            if (currentWord[left] != currentWord[right]) {
                isPalindrome = 0; // Not a palindrome (0 = false)
                break;           // Stop checking this word
            }
            left++;
            right--;
        }

        // If we found a palindrome, save its index and stop searching the array
        if (isPalindrome == 1) {
            foundIndex = i;
            break;
        }
    }

    // Print the result based on whether a palindrome was found
    if (foundIndex != -1) {
        printf("Output: \"%s\\n\", words[foundIndex]);
    } else {
        printf("Output: \"\\n\\n");
    }

    return 0;
}
```

Output



```
Output: "ada"
```

```
=== Code Execution Successful ===
```

Complexity Analysis

Time Complexity: $O(N * M)$ in the worst case, $O(M)$ in the best case.

Space Complexity: $O(1)$

Result

Thus the program was executed successfully and the output was verified.